

OMI-Lisp Native ASCII CONS Diagram Profile

Status

Canonical user-facing projection specification.

1. Purpose

OMI-Lisp SHALL provide a native ASCII renderer for:

```
atoms
NIL
proper lists
improper dotted pairs
nested lists
shared references
cyclic references
QuQuart registers
tetrahedral CONS centroids
```

The renderer SHALL draw the actual runtime object structure.

It SHALL NOT reconstruct a diagram by guessing from formatted prose.

Canonical flow:

```
OMI-Lisp source
  |
  v
parser
  |
  v
CONS object graph
  |
  +----> dotted notation
  |
  +----> canonical binary serialization
  |
  +----> ASCII box-and-arrow projection
```

2. Authority Boundary

The ASCII diagram is a projection.

It does not replace the canonical source or object identity.

```
Source text parses.  
  
CONS cells carry structure.  
  
Canonical bytes carry identity.  
  
ASCII diagrams expose structure to humans.  
  
Validation accepts.  
  
Receipt records.
```

Moving a box, changing whitespace, or selecting another drawing style **MUST NOT** change the logical identity of the represented CONS structure.

3. Basic CONS Cell

A CONS cell contains two ordered slots:

```
+-----+-----+  
|  CAR  |  CDR  |  
+-----+-----+
```

The CAR is always displayed on the left.

The CDR is always displayed on the right.

The order **MUST NOT** be reversed by the renderer.

A direct dotted pair:

```
(A . B)
```

is rendered:



The compact form is display-only unless a separate compact-diagram parser is explicitly selected.

4. Proper List Rendering

The proper list:

is represented internally as:

Canonical horizontal diagram:



Traditional domino profile:

```

      ---
|   |   |--->|   |   |--->|   |   |---> NIL
      ---
      |       |       |
      v       v       v
    rose   violet buttercup

```

All three views MUST represent the same object graph.

5. Empty List

The empty list is:

()

and canonical symbolically:

NIL

Default ASCII rendering:

NIL

Optional boxed rendering:

```

+-----+
|  NIL  |
+-----+

```

OMI-Lisp SHALL preserve the Lisp law:

NIL is the empty list.

NIL also acts as false in the base truthiness profile.

6. List Containing NIL

The list:

```
(A ())
```

is equivalent to:

```
(A NIL)
```

Its internal structure is:

```
(A . (NIL . NIL))
```

Rendered:

```
+-----+-----+      +-----+-----+
|  CAR  |  CDR  |      |  CAR  |  CDR  |
+-----+-----+      +-----+-----+
|      |      +-----> |      |      |
|      |      |         |      |      |
v         v         v         v
A         NIL        NIL
```

Compact:

```
[A | o]--->[NIL | NIL]
```

The first `NIL` is a list element.

The second `NIL` terminates the enclosing list.

The renderer SHOULD preserve this distinction visually.

7. Improper List

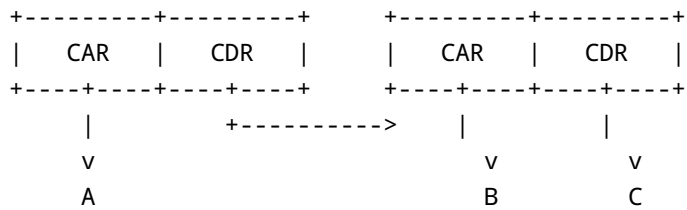
The improper list:

(A B . C)

is represented as:

(A . (B . C))

Rendered:



Compact:

[A | o]--->[B | C]

The final CDR is **C**, not **NIL**.

The renderer MUST make improper termination visible.

Recommended marker:

[A | o]--->[B | .C]

or:

[A | o]--->[B | C] ; improper tail

8. Nested List

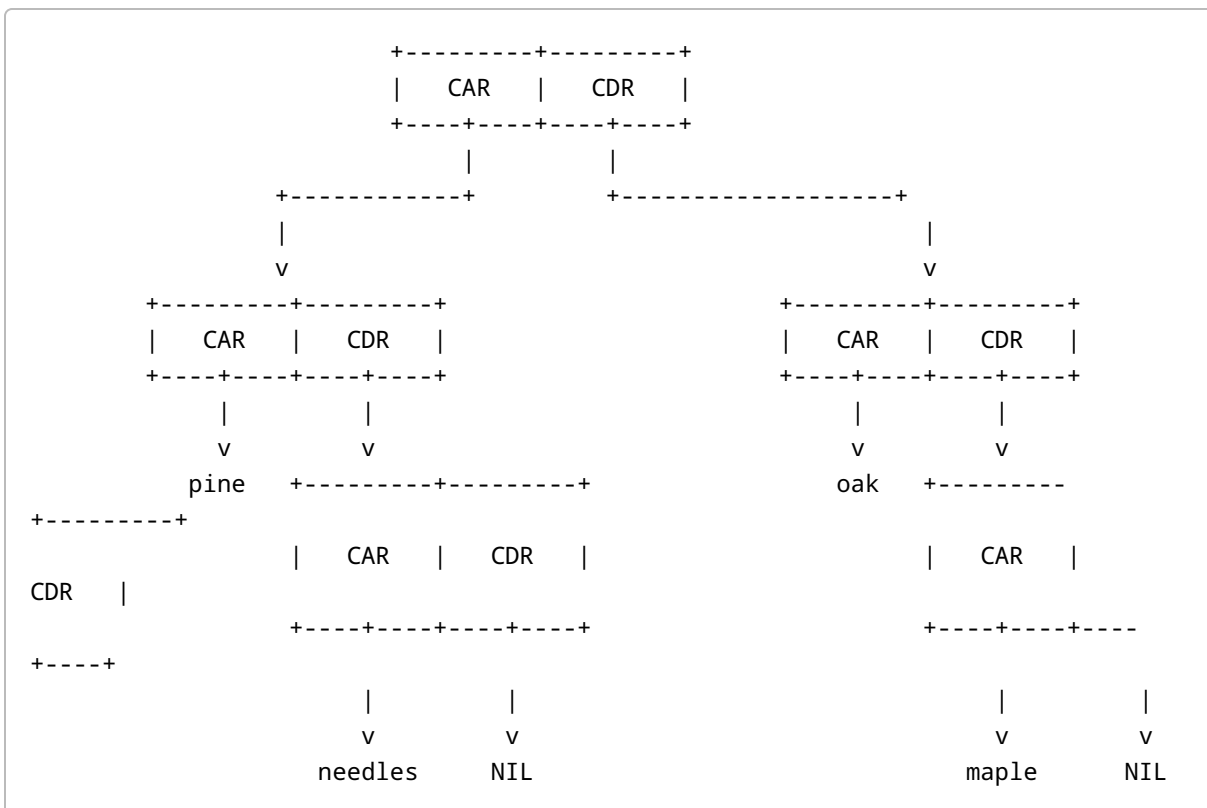
The list:

((pine needles) oak maple)

is internally:

```
((pine . (needles . NIL))
 .
 (oak . (maple . NIL)))
```

Canonical ASCII tree:



Compact nested profile:

```
[o | o]----->[oak | o]---->[maple | NIL]
|
v
[pine | o]---->[needles | NIL]
```

The renderer SHOULD avoid crossing edges where possible.

9. Runtime Object IDs

Every non-atomic object MAY receive a stable render-local identifier:

```
#1  
#2  
#3
```

Example:

```
#1 [rose | #2]  
#2 [violet | #3]  
#3 [buttercup | NIL]
```

Render-local IDs:

```
MUST be deterministic for a given traversal profile  
  
MUST NOT be used as receipt identity  
  
MUST NOT expose raw memory addresses
```

Recommended traversal:

```
depth-first  
  
CAR before CDR  
  
first occurrence assigns ID
```

10. Shared References

Lisp object graphs may contain shared cells.

Example conceptual graph:

```
(define tail '(violet buttercup))  
(cons 'rose tail)  
(cons 'iris tail)
```

Both lists reference the same tail.

The renderer MUST NOT duplicate the shared object as though two independent tails existed.

It should render:

```
#1 [rose | #3]
#2 [iris | #3]
#3 [violet | #4]
#4 [buttercup | NIL]
```

Graph view:

```
[rose | o]-----+
                  |
                  +---->[violet | o]--->[buttercup | NIL]
                  |
[iris | o]-----+
```

Shared references SHOULD use one object ID and multiple incoming arrows.

11. Cyclic CONS Structures

A hosted OMI-Lisp profile MAY permit cycles.

Example:

```
#1 = (A . #1)
```

The renderer MUST detect the cycle.

It MUST NOT recurse forever.

Canonical display:

```
#1 [A | #1]
    ^  |
    +---+
```

Expanded:

```

+-----+-----+
|  CAR  |  CDR  | #1
+-----+-----+
|      |      |
|  v   |      |
|  A   |      | +-----> #1

```

Canonical tree serialization MUST reject cycles unless a graph-serialization profile is selected.

ASCII rendering MAY still display them safely.

12. Native OMI-Lisp Rendering Forms

Required user-facing procedures:

```
(draw value)
```

Returns the default ASCII diagram as a string.

```
(draw value 'compact)
```

Returns compact box notation.

```
(draw value 'domino)
```

Returns traditional two-box CONS cells.

```
(draw value 'tree)
```

Returns a nested tree projection.

```
(draw value 'graph)
```

Returns a graph-aware projection with object IDs and shared references.

```
(print-diagram value)
```

Writes the diagram through the host output capability and returns the value or `NIL`, according to profile.

13. Example OMI-Lisp Session

```
(define flowers '(rose violet buttercup))  
  
(draw flowers 'compact)
```

Output:

```
[rose | o]--->[violet | o]--->[buttercup | NIL]
```

```
(draw flowers 'domino)
```

Output:

```
  ---  ---  ---  
|  |  |--->|  |  |--->|  |  |---> NIL  
  ---  ---  ---  
  |      |      |  
  v      v      v  
rose    violet  buttercup
```

Nested example:

```
(define tree '((pine needles) oak maple))  
  
(draw tree 'graph)
```

Output:

```
#1 [#2 | #4]  
#2 [pine | #3]  
#3 [needles | NIL]  
#4 [oak | #5]  
#5 [maple | NIL]
```

Optional layout:

```
#1 [o | o]----->#4 [oak | o]--->#5 [maple | NIL]
|
v
#2 [pine | o]--->#3 [needles | NIL]
```

14. Diagrams as OMI DataViews

The renderer SHOULD produce an intermediate DataView graph before emitting characters.

```
typedef enum omi_diagram_node_kind {
    OMI_DIAGRAM_CONS = 0,
    OMI_DIAGRAM_ATOM,
    OMI_DIAGRAM_NIL,
    OMI_DIAGRAM_REFERENCE
} omi_diagram_node_kind;

typedef struct omi_diagram_node {
    uint32_t id;
    omi_diagram_node_kind kind;

    omi_value source_value;

    uint32_t car_target;
    uint32_t cdr_target;

    int32_t x;
    int32_t y;

    uint32_t width;
    uint32_t height;
} omi_diagram_node;

typedef struct omi_diagram_edge {
    uint32_t from_node;
    uint8_t from_port;

    uint32_t to_node;

    bool cycle;
    bool shared;
} omi_diagram_edge;

typedef struct omi_diagram {
```

```
    omi_diagram_node *nodes;
    size_t node_count;

    omi_diagram_edge *edges;
    size_t edge_count;
} omi_diagram;
```

Pipeline:

```
runtime value
→
object-graph discovery
→
diagram graph
→
layout
→
ASCII raster
```

This allows the same diagram graph to be projected later as:

```
ASCII

Unicode box drawing

JSON Canvas

SVG

terminal UI

HTML

Figma/FigJam

Graphviz
```

without changing Lisp semantics.

15. ASCII Raster

The ASCII renderer SHOULD operate on a character grid:

```
typedef struct omi_ascii_canvas {
    char *cells;
    size_t width;
    size_t height;
} omi_ascii_canvas;
```

Required operations:

```
omi_status omi_ascii_put(
    omi_ascii_canvas *canvas,
    size_t x,
    size_t y,
    char character
);

omi_status omi_ascii_text(
    omi_ascii_canvas *canvas,
    size_t x,
    size_t y,
    const char *text
);

omi_status omi_ascii_horizontal(
    omi_ascii_canvas *canvas,
    size_t x0,
    size_t x1,
    size_t y
);

omi_status omi_ascii_vertical(
    omi_ascii_canvas *canvas,
    size_t x,
    size_t y0,
    size_t y1
);

omi_status omi_ascii_arrow_right(
    omi_ascii_canvas *canvas,
    size_t x,
    size_t y
);

omi_status omi_ascii_arrow_down(
    omi_ascii_canvas *canvas,
    size_t x,
```

```
size_t y  
);
```

The initial profile SHOULD use only ASCII bytes:

```
0x20..0x7E
```

Recommended characters:

```
+ - |  
> v ^  
o .  
[ ]  
#
```

Unicode box drawing MAY be an optional display profile.

16. Layout Algorithm

The base layout SHOULD follow these rules:

1. Discover all reachable objects.
2. Assign deterministic render IDs.
3. Mark shared and cyclic references.
4. Compute subtree widths.
5. Place the root at the upper-left or top-center.
6. Place proper-list CDR chains horizontally.
7. Place nested CAR structures below their parent.
8. Reserve one blank row between levels.
9. Route CAR arrows downward.
10. Route CDR arrows rightward.
11. Replace repeated structures with references.

12. Expand the canvas if a label is wider than its cell.

Proper list optimization:

```
CDR chain  
→ horizontal backbone  
  
nested CAR pair  
→ downward branch
```

This reproduces familiar Lisp diagrams naturally.

17. Stable Label Formatting

Atoms MUST be formatted using the normal OMI-Lisp printer.

Examples:

```
symbol  
rose  
  
integer  
42  
  
hexadecimal profile  
0xAA55  
  
string  
"hello"  
  
boolean  
#t  
  
empty list  
NIL
```

Long labels SHOULD be handled by:

```
expanding the box
```


- wrapping under a declared width
- or replacing the label with a render-local reference

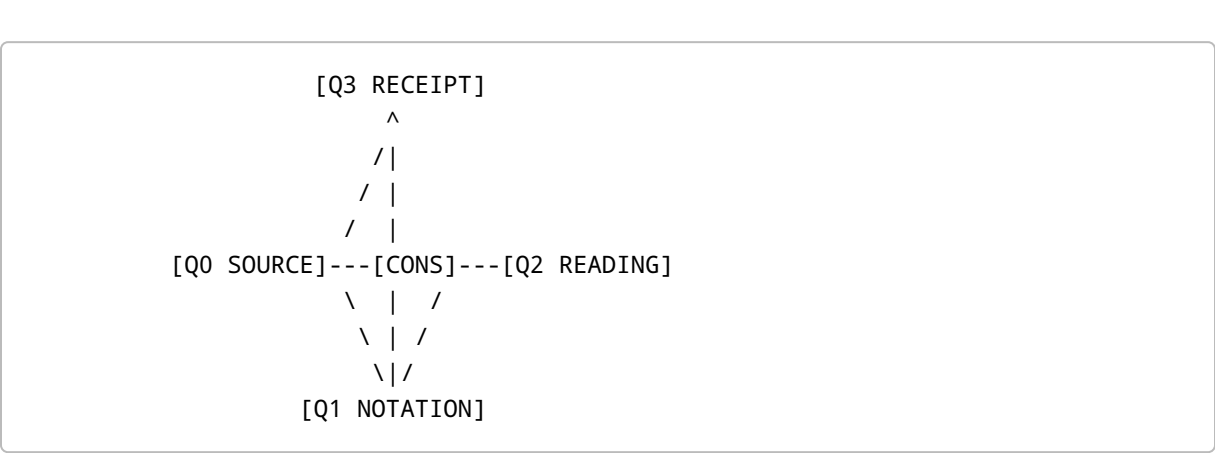
The renderer MUST NOT silently truncate labels in canonical test mode.

18. QuQuart Diagram

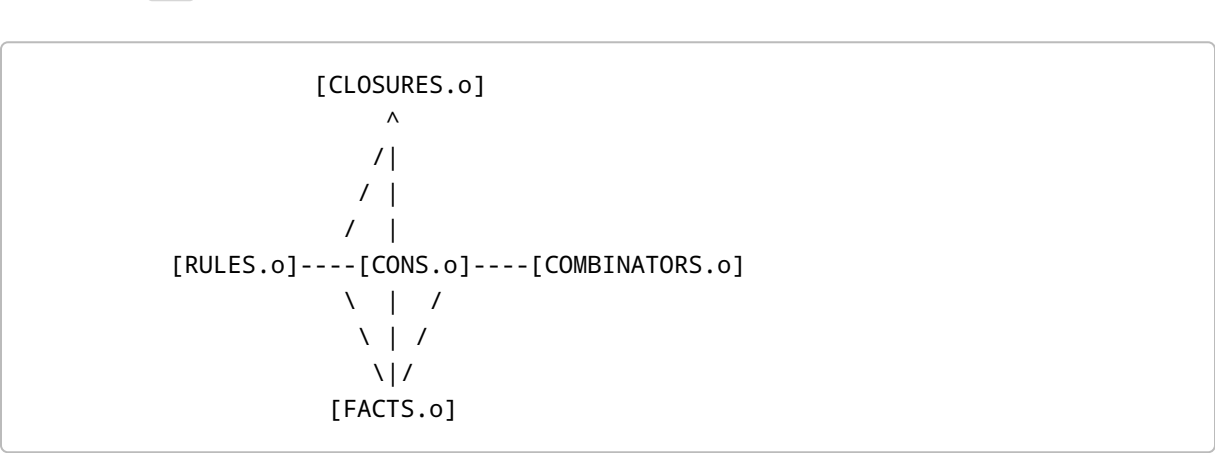
The QuQuart can be rendered natively:

```
(draw
  (ququart source notation reading receipt)
  'tetrahedral)
```

Output:



Or using the roots:



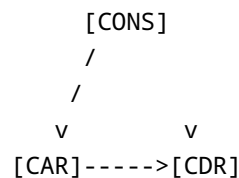
This diagram is derived from the tetrahedral centroid value.

It is not separately hand-authored authority.

19. Quasigroup Triple Diagram

```
(draw
  (quasigroup-triple car cdr cons)
  'triangle)
```

Output:



CAR * CDR = CONS
CAR \ CONS = CDR
CONS / CDR = CAR

A reversible resolver diagram SHOULD distinguish all three directions.

20. Truth-Table CONS Diagram

For rank two:

```
(draw
  (truth4-cell 0x6)
  'truth-cons)
```

Possible output:

CDR truth table: XOR = 0x6 = FTF

CAR		CDR lookup		CONS
+-----+		+-----+		+-----+
TT	---->	bit 3: F	---->	F
TF	---->	bit 2: T	---->	T
FT	---->	bit 1: T	---->	T

```
| FF |---->| bit 0: F |---->| F |  
+-----+ +-----+ +-----+
```

This directly visualizes:

```
CAR assignment  
CDR truth-table row  
CONS result
```

21. Diagram Receipts

By default, receipts **MUST** bind the underlying canonical structure, not whitespace layout.

A projection receipt **MAY** additionally bind:

```
diagram profile  
renderer version  
canvas dimensions  
ASCII output digest
```

The two receipt classes **MUST** remain distinct:

```
semantic receipt  
projection receipt
```

A different diagram width may change the projection receipt while leaving the semantic receipt unchanged.

22. Parsing ASCII Diagrams

The first implementation **SHOULD** treat diagrams as output only.

OMI-Lisp source
→ ASCII diagram

A later profile MAY define machine-readable ASCII diagrams.

That profile would require:

exact box dimensions

exact port markers

object IDs

unambiguous arrow grammar

escaped atom labels

cycle notation

shared-reference notation

Until that grammar is formally specified:

ASCII diagrams are native projections,
not canonical input syntax.

The dotted OMI-Lisp notation remains the canonical textual input.

23. C API

```
typedef enum omi_diagram_profile {  
    OMI_DIAGRAM_COMPACT = 0,  
    OMI_DIAGRAM_DOMINO,  
    OMI_DIAGRAM_TREE,  
    OMI_DIAGRAM_GRAPH,  
    OMI_DIAGRAM_TETRAHEDRAL,  
    OMI_DIAGRAM_TRUTH_CONS  
} omi_diagram_profile;  
  
typedef struct omi_diagram_options {  
    omi_diagram_profile profile;
```

```

uint32_t maximum_width;
uint32_t maximum_height;

bool show_object_ids;
bool show_slot_labels;
bool show_types;
bool show_shared_references;
bool ascii_only;
} omi_diagram_options;

omi_status omi_draw_ascii(
    omi_runtime *runtime,
    omi_handle value,
    const omi_diagram_options *options,
    char *buffer,
    size_t capacity,
    size_t *required,
    omi_error *error
);

```

The standard two-call buffer pattern applies:

```

call with NULL buffer to obtain required size

allocate buffer

call again to render

```

24. Required Tests

Proper lists

```
(rose violet buttercup)
```

MUST produce three CONS cells and a final `NIL`.

Empty list

```
()
```

MUST produce `NIL`.

Nested list

```
((pine needles) oak maple)
```

MUST preserve the downward CAR branch and horizontal outer CDR chain.

Improper pair

```
(A . B)
```

MUST display `B` as a direct CDR target, not as a list terminator.

Shared tail

Two roots sharing one tail MUST render one tail object with multiple incoming edges.

Cycle

A cycle MUST terminate rendering using an object reference.

CAR/CDR order

```
(A . B)
```

and:

```
(B . A)
```

MUST produce visibly different diagrams.

Determinism

The same structure and rendering profile MUST produce identical ASCII bytes.

Semantic independence

Changing line wrapping MUST NOT change the semantic digest.

25. User-Facing Canon

OMI-Lisp dotted notation is the machine-readable relation.

The CONS object graph is the executable structure.

The ASCII diagram is the native human-readable DataView.

CAR is always the left slot.

CDR is always the right slot.

Arrows preserve actual object references.

Proper lists end in NIL.

Improper lists end in a non-NIL CDR.

Nested lists branch through CAR.

List continuation proceeds through CDR.

Shared structures are drawn once and referenced many times.

Cycles are labeled and never expanded infinitely.

The same runtime graph can project as:

dotted Lisp
ASCII boxes
JSON Canvas
SVG
Graphviz
or another DataView.

Projection does not accept state.

Validation accepts.

Receipt records.

26. Final Compact Form

OMI-Lisp reads the dot.

CONS builds the graph.

CAR branches downward.

CDR continues rightward.

NIL closes the list.

ASCII draws the graph.

Canonical bytes identify it.

Validation accepts it.

Receipt preserves it.